

SafeHer AI — Women Safety Intelligence System

Complete Project Explanation & Architecture Document

1. Project Overview & Problem Statement

Problem Statement: Women's safety remains a critical global issue. Existing safety applications often require multiple manual interactions to trigger alerts, lacking automated danger detection, real-time stress analysis, and predictive routing to avoid dangerous zones.

Project Overview: SafeHer AI is a state-of-the-art, AI-powered women's safety platform. It integrates a React Native mobile app with a FastAPI backend and a React.js Admin Dashboard. It features instant SOS (Shake/Voice), live GPS telemetry, GPT-4 threat analysis, Whisper AI speech-to-text, acoustic stress detection (Librosa), and Twilio emergency dispatch.

2. Objectives

- Provide an instant, low-friction SOS mechanism via voice, physical shake, or single-tap.
- Automatically detect threats using advanced NLP (GPT-4) and voice stress analysis (Whisper AI + Librosa).
- Provide predictive safe-routing to bypass historical danger zones.
- Establish a high-priority, real-time WebSocket communication channel with centralized dispatch (Admin Dashboard) and trusted contacts.

3. Complete Technology Stack

- **Frontend Mobile:** React Native, Expo, React Navigation, Axios, Socket.IO Client.
- **Frontend Web (Admin):** React.js, Vite, TailwindCSS, Socket.IO Client.
- **Backend:** Python, FastAPI, Uvicorn, Socket.IO (python-socketio), Pydantic.
- **AI/ML:** OpenAI GPT-4 (Text/NLP Analysis), OpenAI Whisper (Speech-to-Text), PyTorch, Librosa (Acoustic Stress Analysis).
- **Database:** MongoDB Atlas, Motor (Async Python Driver).
- **External APIs:** Twilio (SMS/Voice Calls), Google Maps / Mapbox (Telemetry & Routing).

4. System Architecture

SafeHer AI operates on a microservices-inspired monolithic backend. The Mobile app acts as a telemetry and data collection node, streaming GPS coordinates and audio recordings. The FastAPI backend orchestrates AI processing, database persistence, and WebSocket broadcasting. The Admin Web App serves as a live command center receiving alerts.

5. Frontend & Backend Architecture

- **Frontend (Mobile):** Component-based architecture. Features `MapScreen` for navigation and `SOSScreen` for voice/stress trigger. Uses centralized state for JWT management.
- **Backend (FastAPI):** Layered architecture:
 - **API Layer:** `app/api/routes` (e.g., `ai.py`, `sos.py`, `location.py`).
 - **Service Layer:** `app/services` (e.g., `alert_service.py`, `twilio_service.py`, `user_service.py`).
 - **AI Engine:** `app/ai/llm_engine.py`, `voice_analyzer.py`.
 - **Data Layer:** `app/models` (Pydantic models mapping to MongoDB schemas).

6. Complete End-to-End Workflow

1. User travels and app streams GPS data to `/api/location/update`.
2. An emergency is detected (Voice SOS, Shake, or Deviation).
3. Mobile app records audio and sends to `/trigger-voice`.
4. Backend uses Whisper to transcribe audio and Librosa to detect panic.
5. GPT-4 analyzes the transcript. If `danger_level` is HIGH/CRITICAL, an SOS is triggered.
6. `alert_service.py` saves alert to MongoDB.
7. `twilio_service.py` dispatches SMS and Calls to primary/secondary contacts.
8. `socket_manager.py` broadcasts a high-priority alert to the Admin Dashboard.

7. Major Modules & Functionality

- **Auth Module:** JWT-based user and admin authentication (`auth.py`).
- **Location & Telemetry Module:** Live GPS streaming, geofencing, and behavioral deviation monitoring (`location.py`).
- **AI Engine Module:** GPT-4 context evaluation and Whisper transcriptions (`ai.py`).
- **SOS & Alert Module:** Triggering alerts and notifying emergency contacts (`sos.py`, `alert_service.py`).
- **Admin Dashboard Module:** Real-time heatmaps, active alerts list, user analytics (`admin.py`).

8. Algorithms and Techniques Used

8.1. Haversine Algorithm

1. **Name:** Haversine Formula.
2. **Purpose:** Calculate the great-circle distance between two GPS coordinates.
3. **Where Used:** GPS tracking, geofencing danger zones, and behavioral deviation monitoring.
4. **Input:** (Lat1, Lon1) and (Lat2, Lon2).
5. **Step-by-step:** Convert coordinates to radians. Apply formula: $a = \sin^2(\Delta lat/2) + \cos(lat1) * \cos(lat2) * \sin^2(\Delta lon/2)$. $c = 2 * \text{atan2}(\sqrt{a}, \sqrt{1-a})$. Distance = $R * c$.
6. **Output:** Distance in meters/kilometers.
7. **Pseudocode:**

```
def haversine(lat1, lon1, lat2, lon2):  
    R = 6371 # Earth radius  
    dlat, dlon = radians(lat2 - lat1), radians(lon2 - lon1)  
    a = sin(dlat/2)**2 + cos(radians(lat1)) * cos(radians(lat2)) * sin(dlon/2)**2  
    return R * 2 * atan2(sqrt(a), sqrt(1 - a))
```

8. **Time & Space Complexity:** $O(1)$ Time, $O(1)$ Space.
9. **Advantages:** Highly accurate for spherical Earth distance calculations.
10. **Limitations:** Assumes Earth is a perfect sphere, slight inaccuracies for extreme precision.

8.2. AI Predictive Route Safety Engine

1. **Name:** Weighted A* Search (Pathfinding).
2. **Purpose:** Predict the safest route bypassing high-risk danger zones.
3. **Where Used:** MapScreen.tsx and predict-route-safety API.
4. **Input:** Start node, End node, Graph of nodes weighted by crime heatmaps and lighting data.
5. **Step-by-step:** Assign high cost/weights to edges intersecting danger zones. Execute A* search minimizing total cost.
6. **Output:** Array of safe GPS coordinates.
7. **Pseudocode:**

```
def safe_route(start, end, zones):  
    # weight paths based on zone proximity  
    # run A* minimizing risk + distance  
    return optimal_path
```

8. **Time & Space Complexity:** $O(E + V \log V)$ Time, $O(V)$ Space.
9. **Advantages:** Balances path length with safety.
10. **Limitations:** Depends on the accuracy of heatmap data.

8.3. Acoustic Voice Stress Analysis (Librosa)

1. **Name:** Pitch and Energy Thresholding.
2. **Purpose:** Detect panic, screaming, or stress in audio recordings.
3. **Where Used:** voice_analyzer.py.
4. **Input:** .wav audio file.
5. **Step-by-step:** Extract Zero Crossing Rate, MFCCs, and pitch contours using Librosa. Check if pitch variation and energy exceed trained stress thresholds.
6. **Output:** Stress probability score (0.0 to 1.0).
7. **Pseudocode:**

```
y, sr = librosa.load(audio)  
pitch = librosa.piptrack(y=y, sr=sr)  
stress_score = normalize(max(pitch))  
return stress_score > 0.8
```

8. **Time & Space Complexity:** $O(N)$ Time (N = audio samples).
9. **Advantages:** Detects danger even if words are unintelligible.
10. **Limitations:** Can trigger false positives in noisy environments.

9. AI/ML Workflow & NLP Workflow

1. **Audio Capture:** User triggers SOS or speaks a wake word.
2. **Whisper Transcription:** Converts audio to text.
3. **Context Injection:** Surrounding data (GPS, time of day) is appended to the transcript.
4. **LLM Evaluation:** GPT-4 receives a strict system prompt to output JSON classifying danger_level (LOW, MEDIUM, HIGH, CRITICAL), emotion, and trigger_emergency.
5. **Action:** Backend parses JSON. If trigger_emergency is true, dispatch logic runs.

10. Emergency Severity/Risk Classification Logic

The GPT-4 engine classifies risk into:

- **LOW:** Suspicious situation (e.g., "Someone is walking behind me"). Action: Log, alert local contacts silently.
- **MEDIUM:** Unsafe condition (e.g., "I feel scared"). Action: Warning push notifications.
- **HIGH:** Emergency likely (e.g., "Stop following me!"). Action: SOS SMS dispatched.
- **CRITICAL:** Immediate danger (e.g., "Help!"). Action: Twilio Voice Call, Police Dashboard Alert.

11. GPS Tracking & Route Deviation Workflow

- User initiates a "Trip". trip_history record is created.
- Mobile app posts location to /api/location/update every X seconds.
- Backend checks distance from the planned_route.
- If distance > Threshold (e.g., 50 meters) or user stops for > 5 mins, a "Behavioral Deviation" is flagged.
- Risk score scales to 89%, prompting a user check-in. If ignored, triggers SOS.

12. Security & Authentication Architecture

- **JWT Authentication:** POST /api/auth/login issues an Access Token (signed with HS256).
- **Password Hashing:** bcrypt is used to salt and hash passwords before MongoDB insertion.
- **RBAC & Permissions:** Two primary roles: USER and ADMIN. Admins have exclusive access to /api/admin/alerts/active and heatmap configurations.
- **Socket Security:** Socket connections require JWT validation in the handshake.

13. Database Design and Schemas (MongoDB)

- **Users:** _id, name, email, hashed_password, role.
- **Contacts:** user_id, name, phone_number, is_primary.
- **Alerts (SOS):** _id, user_id, location (Lat/Lon), status (active/resolved), danger_level, ai_transcript.
- **DangerZones:** name, location, radius, risk_score.
- **TripHistory:** user_id, start_location, planned_route (Array of Lat/Lon), deviations_detected.

14. Real-time Communication (Socket.IO)

`socket_manager.py` manages active connections. Mobile clients emit `update_location`. The server broadcasts `sos_alert` events to Admin clients instantly, updating the live React dashboard without page reloads.

15. Twilio SMS and Voice Call Workflow

Handled by `twilio_service.py`. On Critical SOS, Twilio API creates a programmable SMS to contacts ("EMERGENCY: [Name] needs help at [Google Maps Link]") and initiates a TwiML Voice Call to the primary contact playing an automated text-to-speech warning.

16. Complete API Documentation

Endpoint	Method	Description
/api/auth/register	POST	Registers user, hashes password.
/api/auth/login	POST	Authenticates, returns JWT.
/api/sos/trigger	POST	Triggers manual SOS, fires Socket event.
/api/sos/trigger-voice	POST	Uploads audio, runs Whisper/GPT-4/Librosa, triggers AI SOS.
/api/location/update	POST	Stores live GPS, checks route deviation.
/api/ai/predict-route-safety	POST	Returns safe coordinates bypassing danger_zones.
/api/admin/alerts/active	GET	Admin only. Lists unresolved SOS incidents.

17. Error Handling & Testing

- Error Handling:** FastAPI `HTTPException` used globally. Pydantic validates payload schemas, returning 422 Unprocessable Entity for bad data.
- Testing:** Uses `pytest` (e.g., `test_api_stats.py`, `test_registration.py`) for API unit tests. `test_db.py` ensures MongoDB connectivity.

18. Deployment Architecture, Performance, Scalability

- Backend:** Gunicorn wrapping Uvicorn workers for async processing. Deployed on AWS EC2 or Heroku.
- Database:** MongoDB Atlas (Serverless/Dedicated cluster).
- Frontend:** Expo EAS for OTA updates and App Store builds. Admin dashboard hosted on Vercel/Netlify.
- Scalability:** Stateless JWT design allows horizontal scaling of FastAPI nodes behind a load balancer. MongoDB replica sets handle heavy read/write GPS telemetry.

19. Limitations & Future Enhancements

- Limitations:** Requires active internet connection for GPT-4/Whisper APIs. High battery drain due to continuous GPS tracking.
- Future Enhancements:** Offline AI processing using on-device models (e.g., Llama.cpp). Hardware integration with smart jewelry/wearables.

20. Complete Real-World Use Case

Scenario: A user is walking home at night. She enters her destination into the SafeHer Map. The AI routes her away from a poorly lit park (marked High Risk). Halfway, a stranger approaches aggressively. She triggers the Voice SOS: "Leave me alone, help!".

Execution:

- Phone records audio, sends to `/trigger-voice`.
- Whisper transcribes "Leave me alone, help!".
- Librosa detects high pitch/stress. GPT-4 flags CRITICAL danger.
- Twilio calls her primary contact and texts her location.
- Admin Dashboard siren blares, showing her live blinking GPS dot.

Part 2: 50 Project-Specific Viva Questions and Answers

Q1. What is the primary tech stack of SafeHer AI?

A1. React Native (Mobile), React.js (Admin), FastAPI (Python Backend), MongoDB, Socket.IO, OpenAI (GPT-4, Whisper).

Q2. Why was FastAPI chosen over Django or Flask?

A2. FastAPI supports asynchronous programming natively (`async/await`), which is crucial for handling WebSockets and concurrent AI API calls rapidly.

Q3. How is the database structured?

A3. We use MongoDB Atlas (NoSQL) because GPS tracking (GeoJSON) and dynamic AI analysis payloads require flexible, schema-less document storage.

Q4. What algorithm is used to calculate the distance between the user and a danger zone?

A4. The Haversine formula, which calculates the great-circle distance between two points on a sphere.

Q5. How does the AI Route Safety Prediction work?

A5. It utilizes a pathfinding algorithm (like weighted A*) where nodes within `danger_zones` are assigned extremely high traversal costs, forcing the route generator to bypass them.

Q6. What does OpenAI Whisper do in this project?

A6. It transcribes the raw audio captured during a voice SOS trigger into text, which is then fed into GPT-4.

Q7. How do you analyze stress in the user's voice?

A7. Using the `librosa` Python library to extract acoustic features (pitch, energy, MFCCs) to detect panic or screaming independently of the spoken words.

Q8. What happens if the GPT-4 API fails during an emergency?

A8. The system falls back to a deterministic keyword matching algorithm (e.g., checking for "help", "save me") to ensure the SOS is still triggered.

Q9. How are passwords secured in the database?

A9. They are salted and hashed using `bcrypt`. Plain text passwords are never stored.

Q10. Explain the role of JWT in this project.

A10. JSON Web Tokens are used for stateless authentication. Upon login, the user gets a JWT, which is sent in the `Authorization` header for protected API routes and Socket handshakes.

Q11. What is Role-Based Access Control (RBAC) here?

A11. Users and Admins have different roles. The Admin Dashboard APIs validate the JWT role to ensure only authorized personnel can view global alerts.

Q12. How does the live GPS tracking feature update the Admin Dashboard?

A12. The mobile app sends GPS coordinates over a persistent Socket.IO connection. The backend receives this and immediately broadcasts it to connected Admin clients.

Q13. Why use WebSockets instead of HTTP Polling for tracking?

A13. WebSockets maintain a persistent, bidirectional connection, reducing latency to milliseconds and eliminating HTTP header overhead, making it ideal for real-time tracking.

Q14. How does Twilio integration work?

A14. The `twilio_service.py` uses the Twilio REST API with our Account SID and Auth Token to dispatch programmable SMS and initiate TwiML voice calls to emergency contacts.

Q15. What are danger_zones in the MongoDB schema?

A15. A collection storing high-risk areas with a latitude, longitude, radius, and `risk_score`, used for geofencing warnings.

Q16. How do you detect "Behavioral Deviation"?

A16. By continuously comparing the user's live GPS coordinates against the `planned_route` array. If the Haversine distance exceeds a threshold, an alert is flagged.

Q17. What is the time complexity of the Haversine formula?

A17. $O(1)$, as it involves a fixed number of mathematical operations regardless of dataset size.

Q18. What is Pydantic used for in FastAPI?

A18. For data validation and settings management. It ensures that incoming JSON payloads strictly match the expected data types.

Q19. How do you prevent API abuse (Rate Limiting)?

A19. By implementing rate limiters (e.g., using Redis or in-memory caches) on endpoints like `/sos/trigger` to prevent spamming SMS APIs.

Q20. What is the purpose of socket_manager.py?

A20. It manages active Socket.IO connections, handles disconnections, and provides helper methods to broadcast events to specific rooms or roles.

Q21. How does the frontend handle state management?

A21. React Native uses Context API or Redux (depending on implementation) to manage user session, live location state, and active alerts globally.

Q22. What happens if the user loses internet connection?

A22. The app stores location history locally and attempts to send SMS directly via the device's native telephony API as a fallback.

Q23. Explain the GPT-4 System Prompt design.

A23. The prompt explicitly instructs GPT-4 to act as a safety intelligence engine, classify danger into four strict levels (LOW, MEDIUM, HIGH, CRITICAL), and output only valid JSON.

Q24. How is the AI output parsed securely?

A24. We parse the GPT-4 string using Python's `json.loads()` and validate it against a Pydantic model (`DangerLevel` schema) to prevent arbitrary output execution.

Q25. What is the architecture pattern of the Mobile App?

A25. It follows a modular Component-based architecture, separating UI screens, API clients, and native device module integrations (GPS, Microphone).

Q26. How do you ensure the Twilio SMS contains accurate location data?

A26. The backend constructs a Google Maps URL using the latest latitude and longitude retrieved from the user's active session state in MongoDB.

Q27. How does the system handle concurrent SOS requests?

A27. FastAPI runs on ASGI (Asynchronous Server Gateway Interface) using Uvicorn, allowing it to handle thousands of concurrent asynchronous requests without blocking.

Q28. What is the significance of the `_check_ai()` lazy loading in `voice_analyzer.py`?

A28. PyTorch and Whisper are heavy libraries. Lazy loading prevents memory crashes (e.g., `WinError 1455`) during startup by only importing them when an audio file is actually uploaded.

Q29. How is the React Admin Dashboard protected?

A29. Protected routes in React Router check for a valid Admin JWT in local storage before rendering the dashboard components.

Q30. What is the difference between `trigger` and `trigger-voice` APIs?

A30. `trigger` is an instant button-press SOS (deterministic). `trigger-voice` accepts audio, processes it via AI, and dynamically determines the threat level before triggering.

Q31. How is CORS handled in FastAPI?

A31. We use the `CORSMiddleware` in `main.py` to allow requests from the specific React Native and Admin Dashboard origins.

Q32. What is GeoJSON and is it used here?

A32. GeoJSON is a format for encoding geographic data structures. MongoDB uses it for `2dsphere` indexes to efficiently query spatial data (e.g., finding nearby danger zones).

Q33. How do you test the APIs locally?

A33. By running the FastAPI server and navigating to the auto-generated Swagger UI at `/docs`.

Q34. What is Librosa's Zero Crossing Rate used for?

A34. It helps distinguish between harmonic sounds (speech) and noisy/percussive sounds (screams, crashes) during stress analysis.

Q35. Can the admin trace the historical path of an emergency?

A35. Yes, the `trip_history` collection stores an array of coordinates tracking the entire journey.

Q36. How do you deploy the FastAPI backend?

A36. Typically containerized with Docker and deployed on AWS ECS, EC2, or Heroku, fronted by a reverse proxy like Nginx.

Q37. How is environmental noise handled in Voice SOS?

A37. Whisper AI is highly robust to noise, and Librosa filters can be applied to normalize audio before analysis.

Q38. Why use Motor instead of PyMongo?

A38. Motor is an asynchronous Python driver for MongoDB, which integrates perfectly with FastAPI's `async` event loop, unlike synchronous PyMongo.

Q39. What is the purpose of dependencies in FastAPI routes?

A39. Used for injecting reusable logic, such as `get_current_user`, which verifies the JWT before executing the endpoint code.

Q40. How do you secure environment variables?

A40. Keys (OpenAI, Twilio, JWT Secret) are stored in a `.env` file, loaded via `python-dotenv`, and strictly ignored in `.gitignore`.

Q41. What is the fallback if Twilio services are down?

A41. The system alerts the Admin Dashboard via WebSockets and can trigger Firebase Cloud Messaging (FCM) push notifications to contacts' apps.

Q42. How does the Shake-to-SOS work?

A42. The React Native app listens to the device's accelerometer. If the acceleration vector exceeds a high gravity threshold (e.g., 3G), it triggers the SOS API.

Q43. Explain the difference between Primary and Secondary contacts.

A43. Primary contacts receive both Automated Voice Calls and SMS. Secondary contacts receive only SMS and Push Notifications.

Q44. How does the frontend handle continuous location tracking in the background?

A44. By using Expo Background Fetch and TaskManager to periodically awake the app and send location payloads to the backend.

Q45. What happens when an Admin resolves an alert?

A45. The alert `status` in MongoDB is updated to `resolved`, and a Socket event is broadcasted to remove the alert from active heatmaps.

Q46. How do you prevent false positives in Voice SOS?

A46. The GPT-4 prompt requires high confidence for CRITICAL flags, and the Librosa stress score must exceed a strict threshold. "Medium" levels prompt the user before SOS.

Q47. What is the difference between `start_location` and `planned_route`?

A47. `start_location` is the single origin point. `planned_route` is an array of predicted future coordinates generated by the routing engine.

Q48. How do you scale WebSocket connections?

A48. By using a Redis adapter/pub-sub system with Socket.IO, allowing multiple backend server instances to broadcast messages across the cluster.

Q49. What is a "Twiml" Voice call?

A49. Twilio Markup Language (Twiml) is XML used to instruct Twilio to use Text-to-Speech (e.g., `<Say>`) to read the emergency warning when the contact answers the phone.

Q50. How does the project benefit women in real-time?

A50. It removes the cognitive load of unlocking a phone and dialing during an attack. AI handles contextual understanding, routing, and dispatching assistance automatically.